

Exercise_03_basic_producer_consumer

La entrega de la práctica debe ser en formato zip donde debe incluir:

- Funcionalidad de Exercise 03 en formato pdf.
- Ficheros de API Java con las modificaciones.

1.- Escribe el nombre del Topic donde el Producer está enviando los mensajes en la API Java

El nombre del Topic es: quotes-input.

2.- Describe los mensajes qué está enviando el Producer en la API Java.

Los mensajes que está enviando el Producer son: Citas de Shakespeare de la obra "As You Like It". También imprime información de libros, pero esta no la envía.

```
Marcus Orn    Fairy tale    Bison Books    The Curious Incident of the Dog in the Night-Time
True is it that we have seen better days.
Ezequiel Graham    Fiction in verse    ECW Press    Behold the Man
True is it that we have seen better days.
Renee Stamm    Mystery    Chronicle Books    The Waste Land
True is it that we have seen better days.
Mrs. Chu Johnston    Horror    Voyager Books    The Moon by Night
Blow, blow, thou winter wind! Thou art not so unkind as man's ingratitude.
Shannan Wilderman II    Western    Voyager Books    Dance Dance Dance
How bitter a thing it is to look into happiness through another man's eyes!
Ms. Vania Boyle    Metafiction    Virgin Publishing    For a Breath I Tarry
How bitter a thing it is to look into happiness through another man's eyes!
Amira Cartwright    Mythopoeia    Chatto and Windus    The Monkey's Raincoat
Blow, blow, thou winter wind! Thou art not so unkind as man's ingratitude.
```

3.- Describe qué hace el código de WordCountConsumer.

El código de *WordCountConsumer*:

- Lee citas desde el *topic* de entrada de Kafka llamado *quotes-input*.
- Procesa cada mensaje (una cita de Shakespeare), separando las palabras.
- Cuenta cuántas veces aparece cada palabra en total.
- Escribe los resultados (palabra + conteo) en el *topic streams-wordcount-output*.

```
| how | 17
| bitter | 17
| a | 92
| thing | 38
| it | 65
| is | 65
| to | 38
| look | 17
| into | 17
| happiness | 17
| through | 17
| another | 17
| man | 72
| s | 51
| eyes | 17
| i | 22
```

4.- Explica cómo el Consumer obtiene los mensajes según el código de la API Java.

- Se conecta a Kafka en localhost:9092.
- Se suscribe al *topic streams-wordcount-output*, que contiene los pares (palabra y cantidad de veces que aparece).
- Consume continuamente los mensajes que van llegando.
- Imprime por consola cada par clave-valor en el formato: | palabra | número de apariciones

5.- Realiza un pequeño cambio en el código de la API Java de Producer modificando el mensaje que se envía.

Los cambios que he realizado han sido:

- Enviar el mensaje en formato JSON.
- Añadir una función que permite al texto escapar de comillas y caracteres especiales.
- Cambiar la cita de *Shakespeare* por una de *Regreso al Futuro*.

```
public class Producer_01 {

    public static void main(String[] args){
        Faker faker = new Faker();

        Properties properties = new Properties();
        properties.put("bootstrap.servers", "localhost:9092");
        properties.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        properties.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

        KafkaProducer<String, String> kafkaProducer = new KafkaProducer<>(properties);

        try{
            for(int i = 0; i < 100000; i++){
                Thread.sleep(1000);

                String author = faker.book().author();
                String genre = faker.book().genre();
                String publisher = faker.book().publisher();
                String title = faker.book().title();
                String quote = faker.backToTheFuture().quote();

                String message = String.format(
                    "{" +
                    "\"author\": \"%s\", " +
                    "\"genre\": \"%s\", " +
                    "\"publisher\": \"%s\", " +
                    "\"title\": \"%s\", " +
                    "\"quote\": \"%s\"" +
                );
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```
        "}",
        escape(author), escape(genre), escape(publisher), escape(ti-
tle), escape(quote)
    );

    System.out.println("\n" + message);
    kafkaProducer.send(new ProducerRecord<>("quotes-input", message));
}
} catch (Exception e){
    e.printStackTrace();
} finally {
    kafkaProducer.close();
}
}

private static String escape(String input) {
    return input.replace("\\", "\\\\").replace("\n", "\\n");
}
}
```